

The Cyber Context of Software Engineering

WM240-24

Author Student ID: u5602780

Module Tutor: Bharath Sadasivaiah

January 23, 2026

Written report

Word Count: 2056

Contents

1	Introduction	2
1.1	Pre-research and Security Philosophy	2
1.2	Architectural Enforcement of Security Principles	3
2	Implementation	5
2.1	Database Design	5
2.2	Secure Implementation of Search and Filter	6
2.3	Backend API Structure	7
2.4	Frontend Structure	8
3	Critical Reflection and SSCoP Principles	9
3.1	Overview of Development and DevSecOps Integration	9
3.2	Reflection on Development Challenges	9
3.3	Alignment with Software Security Code of Practice (SSCoP)	9
3.4	Critical Evaluation: Strengths and Areas for Improvement	10
3.5	Future Roadmap and Forward Planning	11
A	Basic Credentials, Setup And Codebase Access Information	13
B	Software Security Code of Practice Self-Assessment Template	14
C	Appendix: GAIT Usage	20

1 Introduction

Streeling University Library: Inventory Management System Requirements

The objective of this project is to develop a secure and efficient Inventory Management System (IMS) for the Streeling University Library. The system requires a distinct separation of concerns between general users (students) and administrative staff (librarians/admins).

Key frontend requirements identified for the Minimum Viable Product (MVP) include:

- **Authentication:** A secure Login Page and a Registration Page for students, implementing session management.
- **Main Library Catalogue:** A searchable interface featuring sorting algorithms. To optimise performance and security, the system implements server-side pagination (e.g., 100 results per page) to prevent data dumping attacks.
- **User Dashboard:** A personalised page for students displaying borrowing history, active loans, and due dates.
- **Librarian Panel:** A dedicated dashboard for librarians to view recent acquisitions, with functionality to manage inventory (upload, update, delete books) and maintain supplier and category records.
- **Audit/Log Page:** A restricted view for administrators to monitor system activity and security logs. Features include filtering, log downloading, and querying capabilities to track librarian promotion/demotion and system events.
- **Role-Based Access Control (RBAC):** Advanced permission scoping that strictly adheres to the Principle of Least Privilege, granting access only to the API endpoints and functions required by a specific user role.

1.1 Pre-research and Security Philosophy

As established in the requirements, the core security principles of *Least Privilege*, *Secure Defaults*, and *Defence in Depth* form the foundation of this project. To enforce these, I adopted a "Zero Trust Client" approach. Since React bundles frontend logic into client-accessible files, the frontend is treated as a "glass box" where attackers can inspect API structures. Therefore, all security enforcement and data validation occur strictly on the backend.

Secure code requires high quality; designing for security without prioritising maintainability creates a fragile foundation where complexity obscures vulnerabilities. Consequently, a structured codebase is viewed as a prerequisite for security. While standard software engineering principles guide the development, this system views them through a security lens [3]:

- **KISS (Keep It Simple, Stupid):** Simplicity reduces developer error. However, robustness sometimes requires trade-offs; for instance, stock availability is dynamically recalculated on every transaction rather than stored statically. This computational overhead is accepted to prevent data desynchronisation and ensure physical inventory integrity.
- **DRY (Don't Repeat Yourself) (With Security Caveats):** While code reuse is standard, security necessitates separation of concerns. Distinct logic flows are maintained for privileged (librarian) and unprivileged (student) users. For example, student API responses are explicitly filtered to exclude supplier data, whereas librarians receive full datasets. This deliberate code duplication strictly enforces the Principle of Least Privilege.
- **YAGNI (You Aren't Gonna Need It):** Dead code is a potential attack vector. Any unused functions or endpoints are removed to minimise the system's attack surface.
- **Input Validation:** Adopting a Defence in Depth approach, validation is performed twice: once on the frontend for user feedback, and rigorously on the backend to prevent malformed data persistence.

The following table details the functional and non-functional requirements derived from these goals.

Table 1: Library System Requirements (Functional & Non-Functional)

Requirement (Type and Feature)	BDD Scenario
Functional Catalogue Pagination and Filtering	Given the library catalogue contains more than 5 books When a user requests the catalogue page Then the system returns at most 5 books per page And additional results require an explicit page request.
Functional User Dashboard Information	Given a student is authenticated When the student navigates to their dashboard Then the system displays their active loans and borrowing history.
Functional Relational Data Integrity	Given a librarian creates or updates a book record When the data is persisted Then referential integrity constraints are enforced in the database.
Non-Functional API Data Minimisation	Given a student requests book information When the API returns the response Then supplier and internal stock metadata are excluded.
Non-Functional Zero-Trust Frontend	Given the frontend source code is inspected When an unauthorised user analyses the bundle Then no credentials, secrets, or privileged routes are exposed.
Non-Functional Defence-in-Depth Input Validation	Given malicious input is submitted When the request reaches the backend Then the request is rejected regardless of client-side checks.

1.2 Architectural Enforcement of Security Principles

Figure 1 illustrates the high-level architecture. Unlike a monolithic application where components are tightly coupled, this design explicitly separates the *Frontend* (Client) from the *Backend* (Server) via a strict API boundary to enforce security policies.

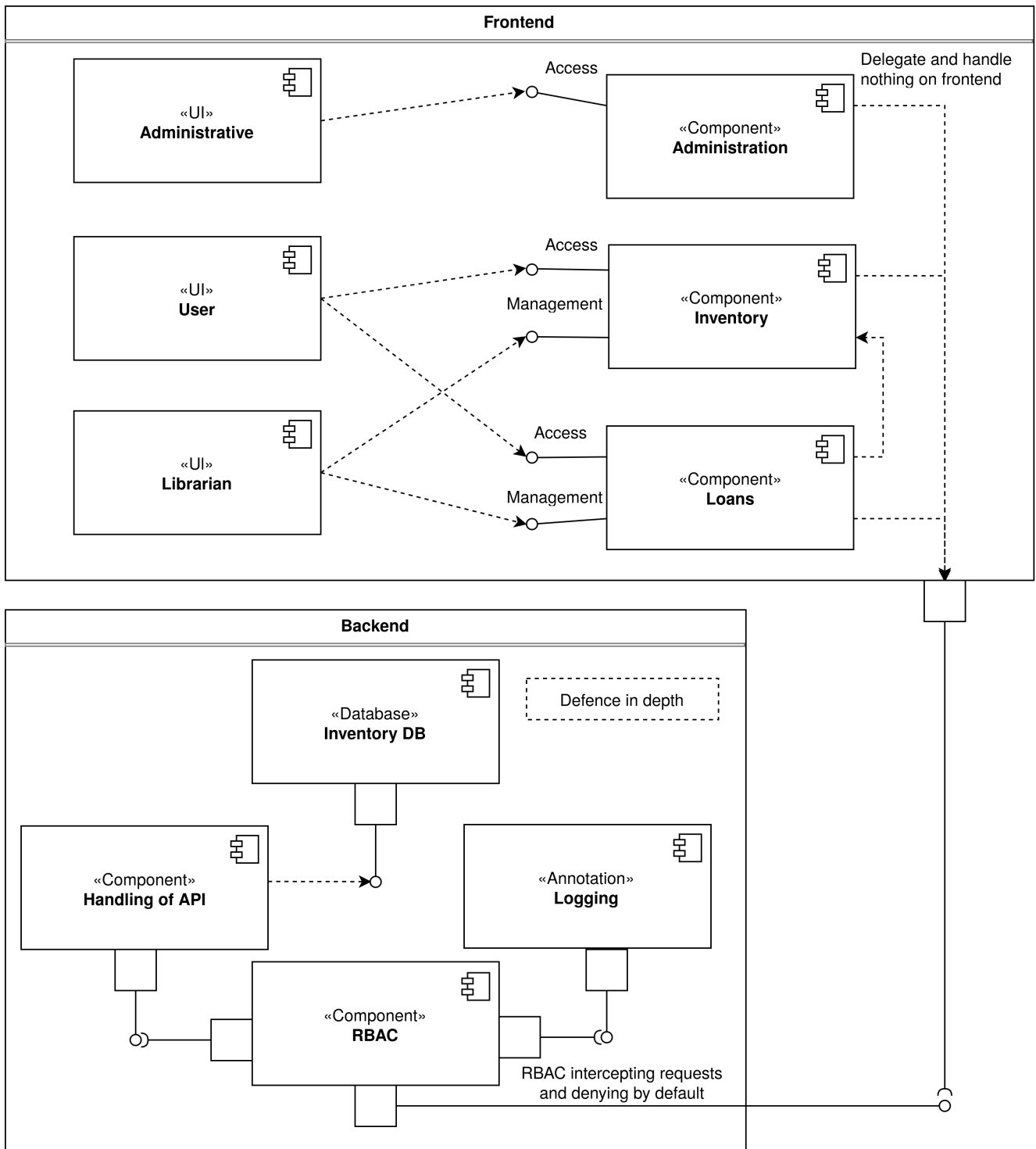


Figure 1: High-Level System Component Diagram

The visual composition of the diagram explicitly implements the required security principles as follows:

- Defence in Depth (Visualised via Layered Boundaries):** As depicted in Figure 1, the architecture enforces a hard boundary between the *Frontend* and *Backend* layers. The **Inventory DB** is isolated deep within the Backend layer, with no direct association lines connecting it to the *Frontend* UI components. All data requests must traverse the **Delegate** interface, then the **Handling of API** component, and finally the specific business logic components. This multi-layered structure ensures that even if the client-side code is compromised, the attacker has no direct path to the database.

- **Least Privilege (Visualised via RBAC Component):** The RBAC component is architecturally positioned as a middleware dependency for the **Handling of API** module. The diagram illustrates that while the **UI User** and **UI Librarian** components initiate requests, they do not possess direct control interfaces to the **Inventory** component. Instead, the **Access** interface strictly mediates these interactions. This ensures that a **Student** user cannot trigger **Inventory Management** functions, as the control flow is intercepted and validated by the RBAC component before execution.
- **Secure Defaults (Visualised via API Handler):** The **Handling of API** component is positioned as the single entry point (Gateway) for all external traffic entering the Backend. By funneling all requests through this central component, the system enforces a "Deny by Default" posture. Any request that does not match a defined route or lacks a valid signature is rejected at this gate before it can interact with the **Loans** or **Inventory** business logic, reducing the attack surface significantly.

2 Implementation

The development followed a DevSecOps methodology, integrating security throughout the lifecycle rather than as an afterthought. The implementation was divided into four distinct phases:

Phase 1: Foundation (Alpha)

The goal was a Proof of Concept (PoC) to validate SQLAlchemy models and API connectivity. This phase implemented basic CRUD functionality using core entities (**User**, **Book**, **Login**) without advanced UI, verifying the fundamental backend architecture.

Phase 2: Robust Expansion (Data Integrity)

The database was expanded to production standards. This involved refactoring for Third Normal Form (3NF) and introducing **Suppliers**, **Book_Categories**, and the critical **Audit_Logs** system to support data integrity and traceability.

Phase 3: Logic Integration & Security Hardening

Complex business logic, specifically the **Search engine** and **Loan Management**, was implemented. Automated Pytest scripts were introduced to verify that RBAC permission scopes correctly enforced "Deny by Default" policies on these new endpoints.

Phase 4: UI Refinement & Acceptance Testing

The final phase focused on frontend security and usability. This included styling for accessibility, visual validation feedback, and integration tests to ensure the React frontend correctly handled token expiration and secure error rendering.

2.1 Database Design

The backend architecture begins with a robust data layer designed in Third Normal Form (3NF) to ensure data integrity.

The schema supports a Role-Based Access Control (RBAC) system for admins, librarians, and students. Physical inventory is managed through a **Books** table, tracking metadata and ISBNs, linked to a **Suppliers** table for procurement records. A many-to-many relationship handles the organisation of books into **Categories**.

Circulation is managed via a **Loans** table, which tracks borrowing history, due dates, and return status. Crucially, the design includes a strict **Audit Logs** table. This captures all user actions, timestamps, and IP addresses, ensuring accountability and enabling the detection of security anomalies, as illustrated in Figure 2.

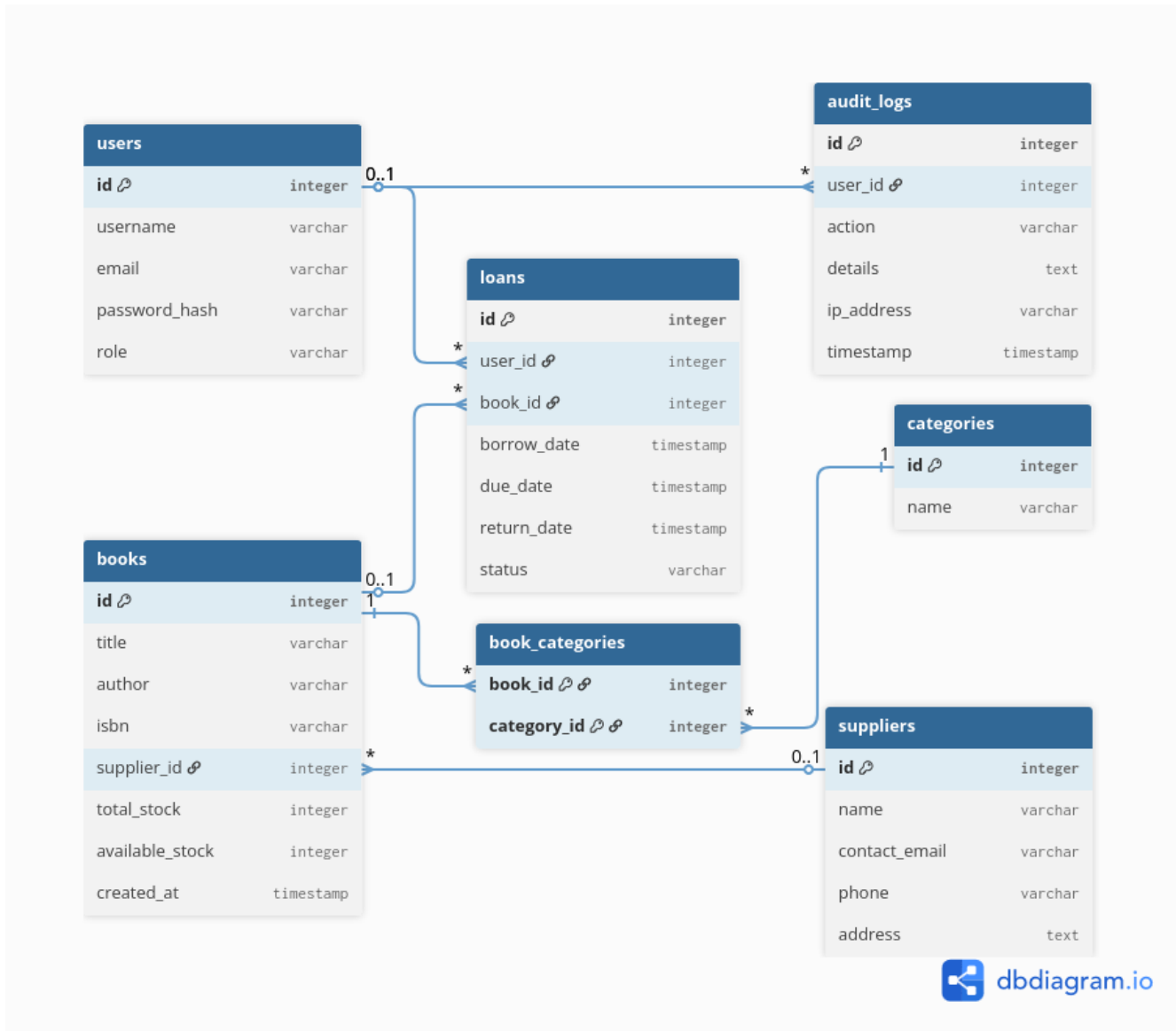


Figure 2: Entity Relationship Diagram (ERD) of the Library Database

2.2 Secure Implementation of Search and Filter

A core functional requirement was the ability to search the catalogue without exposing the database to injection attacks.

The implementation utilises a dynamic query builder on the backend. When a user inputs a search term (e.g., "History"), the frontend dispatches a GET request with query parameters.

- **Backend Logic:** Instead of concatenating strings (which leads to SQL Injection), the API uses SQLAlchemy's ORM filtering. The system checks if the `query` parameter exists and applies a clause. This ensures the database driver handles the escaping of special characters.

```

1  # 3. Apply Filters
2  if search_text:
3      query = query.filter(Book.title.ilike(f"%{search_text}%"))
4
5  if author_filter:
6      query = query.filter(Book.author.ilike(f"%{author_filter}%"))

```

- **Pagination:** To prevent Denial of Service (DoS) via resource exhaustion, the search endpoint enforces server-side pagination. It returns a maximum of 100 records per request, requiring the client to request subsequent pages explicitly.

2.3 Backend API Structure

The backend logic is structured to avoid "God Objects", files that handle too many responsibilities. As the system grew, mixing logic for Books, Loans, and Auth became unmaintainable.

The refactored structure splits the API into specific domains within a `/routes` directory (e.g., `auth.py`, `inventory.py`). The main application file imports these modular blueprints. To ensure consistency, Python decorators are used to standardise error handling and enforce RBAC checks across all endpoints.

```
1 # Back end category handler with error checking and RBAC #
2 @categories_bp.route('/categories', methods=['POST'])
3 @jwt_required()
4 @require_role(LIBRARIAN_ROLE)
5 @handle_db_errors('CATEGORY_CREATE_FAILED')
6 def create_category(current_user):
7     """
8     Creates a new category.
9     Restricted to Librarians.
10    """
11    data = request.get_json() or {}
12
13    # 1. Sanitize & Validate
14    clean_name, error_msg = validate_category_name(data.get('name'))
15    if error_msg:
16        return jsonify(message=error_msg), 400
17
18    # 2. Business Logic: Check duplicates
19    # Case-insensitive check recommended for categories (Sci-Fi == sci-fi)
20    if Category.query.filter(Category.name.ilike(clean_name)).first():
21        return jsonify(message='Category already exists'), 409
22
23    new_category = Category(name=clean_name)
24    db.session.add(new_category)
25    db.session.commit()
26
27    audit.log(
28        user_id=current_user.id,
29        action='CATEGORY_CREATE',
30        details=f"Created category '{new_category.name}' (ID: {new_category.id})"
31    )
32
33    return jsonify(message='Category added successfully', id=new_category.id), 201
34
```

This modularity allows for strict data scoping. The API returns only the necessary information based on the user's role flag. For example, the Admin account is strictly limited to specific high-level functions: 1) onboarding librarians, 2) reviewing audit logs, and 3) managing user roles.

Audit Dashboard

Audit Logs

Summary Stats

User Management

Total Actions (30 Days): 23

Top Actions

- LOGIN_SUCCESS: 7
- LOGOUT: 6
- REGISTER: 3
- CATEGORY_CREATE: 2
- SUPPLIER_CREATE: 1
- ROLE_SWITCH: 1
- LOAN_CREATED: 1
- BOOK_UPDATE: 1
- BOOK_CREATE: 1

Most Active Users

- lib: 12
- admin: 8
- student: 3

Figure 3: Admin Audit Dashboard view

By using `HttpOnly` cookies with `SameSite='Lax'` and enabled CSRF protection, we create a robust defence-in-depth strategy that neutralises Cross-Site Scripting (XSS) by preventing JavaScript from accessing tokens.

2.4 Frontend Structure

The frontend implementation adheres to modern React best practices to ensure a clean, maintainable component hierarchy. The folder structure is organised by feature, separating reusable UI components from page-specific logic. [2, 1]

This structure enhances security through simplified conditional rendering. By breaking pages down into smaller, isolated components, it becomes easier to manage state and control which UI elements are rendered based on the user's authentication status.

```
1 // Main Page Interface //
2 function MainPage() {
3     const { user } = useAuth();
4
5     if (!user) return <div>Loading...</div>;
6
7     // Simple Role Switch
8     return user.role === 'admin'
9         ? <AdminDashboard />
10        : <LibraryDashboard user={user} />;
11 }
```

Frontend API handling includes robust error management, reinforcing the defence-in-depth strategy by mirroring backend validation logic within the user interface.

```
1 {/* Right: Librarian Actions */}
2 {isLibrarian && (
3     <div style={{...styles.section, ...styles.card}}>
4         <h3>Checkout Book</h3>
```

```

5      {book.available_stock > 0 ? (
6          <BorrowForm bookId={book.id} onSuccess={reload} />
7      ) : (
8          <p style={{ color: 'red' }}>Stock unavailable.</p>
9      )}
10  </div>
11  )}
12

```

3 Critical Reflection and SSCoP Principles

3.1 Overview of Development and DevSecOps Integration

This project involved the re-engineering of the Inventory Management System (IMS) for Streeling University Library, shifting from a functional prototype to a security-first application. In alignment with the InfoShield Systems brief, the development lifecycle followed a **DevSecOps** approach, automating security integration at every phase. From design to deployment.

The critical reflection below evaluates the technical challenges encountered during this lifecycle, the system's alignment with the UK Government's Software Security Code of Practice (SSCoP), and a gap analysis of its current security posture.

3.2 Reflection on Development Challenges

The development process highlighted the tension between functional usability and strict security enforcement. Significant technical hurdles were encountered and resolved iteratively:

- **Asynchronous State Management vs. Data Integrity:** A critical race condition occurred when rendering book categories. The frontend attempted to read `item.category_id` before the reference data was fetched, causing render failures. This highlighted the need for defensive coding in the UI layer.

Resolution: The implementation of optional chaining and conditional rendering (`categories.find(...)?.name || "Loading"`) ensured that the UI degrades gracefully rather than crashing, aligning with availability principles.

- **Logic Clarity in Access Control:** Initial authorisation checks utilised complex negative logic (e.g., `if user.role !== LIBRARIAN_ROLE...`). This reduced code readability and increased the probability of logic errors, a critical vulnerability in security contexts.

Resolution: The logic was refactored to use positive assertions (`if role in [ALLOWED_ROLES]`). This change adheres to the principle of "Fail Safe Defaults", ensuring that ambiguity results in denial of access rather than accidental permission.

- **Performance vs. Accuracy Trade-off:** To guarantee stock integrity, the system was designed to recalculate book availability on every transaction rather than relying on cached values. While this introduces computational overhead, it eliminates concurrency bugs where the database count drifts from physical inventory, prioritising **Integrity** over raw performance.

3.3 Alignment with Software Security Code of Practice (SSCoP)

As required by the project brief, the security features implemented in the IMS have been mapped to the principles of the DSIT Software Security Code of Practice.

Table 2: Mapping Security Requirements to SSCoP Principles

Security Requirement and Feature	Description of Implementation	Mapped Principles (SSCoP)
Role-Based Access Control (RBAC)	The system enforces strict separation using backend decorators. It defaults to "Deny All" unless a valid role token is present. This prevents privilege escalation by assuming zero trust.	Principle 1.4: Secure by Default (Claim 1.4.1: Threat modelling used in design) Principle 2.1: Protect Access (Claim 2.1.1: Roles are defined specifying allowed functionality)
Defence-in-Depth Input Validation	Input is validated on the client (for UX) and rigorously sanitised on the server (API layer) before interacting with the SQL database, mitigating SQL Injection and XSS.	Principle 1.4: Secure Design (Claim 1.4.4: Data input to the software is validated) Principle 1.1: Secure Framework (Claim 1.1.6: Software designed for user need and safety)
Immutable Audit Logging	Critical actions (Logins, Inventory Updates) are written to a dedicated <code>AuditLogs</code> table. Timestamps and User IDs provide non-repudiation and forensic capability.	Principle 2.2: Control and Log Changes (Claim 2.2.3: Logs are auditable) Principle 4.3: Incident Management (Claim 4.3.1: Information available to identify incidents)
API Response Filtering (Data Minimisation)	The backend explicitly filters JSON responses based on role. Students receive basic data; Librarians receive supplier info. This prevents data leakage via API inspection.	Principle 1.4: Secure Design (Claim 1.4.5: Sensitive data is securely managed) Principle 3.1: Distribute Securely (Ensuring only intended data reaches the client environment)
Secure Session Management	Implementation of <code>HttpOnly</code> , <code>SameSite=Lax</code> cookies, and CSRF tokens to neutralise session hijacking and Cross-Site Request Forgery.	Principle 1.4: Secure Defaults (Claim 1.4.3: Default passwords/insecure settings are not used) Principle 1.3: Testing (Claim 1.3.1: Testing covers security requirements)

3.4 Critical Evaluation: Strengths and Areas for Improvement

While the IMS successfully implements "Security by Design," a critical gap analysis against the SSCoP standards reveals areas requiring improvement.

Table 3: Project Evaluation against SSCoP Standards

Assessment	Analysis
Key Strengths	• Secure Defaults (Principle 1.4): The application does not rely on obscure logic; it explicitly denies access at the API root level unless authorised. • Data Minimisation: By filtering API responses on the server side, the system avoids "Security by Obscurity," ensuring sensitive fields (e.g., Supplier ID) never leave the server boundary. • Build Integrity (Principle 1.2): The use of Git for version control ensures all code changes are attributable, providing a clear audit trail for the software supply chain.
Areas for Improvement	• Encryption at Rest (Gap in Principle 1.4.5): While passwords are hashed, the database storage itself is not encrypted. A physical server compromise could expose supplier records. • Vulnerability Disclosure (Gap in Principle 3.2): The project lacks a 'security.txt' or formal mechanism for reporting vulnerabilities, which is standard for government-grade software. • Automated SBOM (Gap in Principle 1.2): There is currently no automated Software Bill of Materials (SBOM) generation to track dependency vulnerabilities in real-time.

3.5 Future Roadmap and Forward Planning

To address the identified gaps and prepare the IMS for a production environment, the following roadmap is proposed. This plan prioritises tasks based on the "Defence in Depth" principle.

1. **Infrastructure Hardening:** Implement SSL/TLS to encrypt data in transit and enable Transparent Data Encryption (TDE) for the database to secure data at rest.
2. **Network Segmentation:** Split the API into public (Student) and private (Admin) interfaces. Restrict the Admin API to an internal IP whitelist or VPN access to reduce the attack surface.
3. **Resilience Measures:** Implement strict rate limiting and DDoS protection at the gateway level to prevent availability attacks.
4. **Authentication Maturity:** Upgrade the current session-based auth to include Multi-Factor Authentication (MFA) for Librarian accounts, mitigating the risk of credential theft.

References

- [1] Elif Karadağ. Modern reactjs security guide: Updated best practices for 2025 (with code examples), August 2025. URL <https://medium.com/@elifkaradag100/modern-reactjs-security-guide-updated-best-practices-for-2025-with-code-examples-31f1> Medium article.
- [2] Pulak Pramanik. Best folder structure for full stack web development, aug 2025. URL <https://www.newstorial.com/best-folder-structure-full-stack-web-development/>. Accessed 23 Jan. 2026.
- [3] Anu Upadhyay. 7 common programming principles that every developer must follow, July 2025. URL <https://www.geeksforgeeks.org/blogs/7-common-programming-principles-that-every-developer-must-follow/>. Last updated 12 July 2025.

A Basic Credentials, Setup And Codebase Access Information

```
1 code is stored on the univercitie's gitlab server
2 https://git-lab.cyber.warwick.ac.uk/ccse5602780/ccse-cw1-ims
3 This information is from readme.txt
4
5 # to start backend
6 cd backen
7 source ../venv/bin/activate
8 python3 app.py
9
10 # to start frontend
11 npm start
12
13 # repo comes with already setup:
14 # 1 user, 1 admin, 1 librarian
15 # database and some logs as proof of work
16
17 admin credentials:
18 email: admin@gmail.com
19 password: adminadmin
20
21 librarian credentials:
22 email: lib@gmail.com
23 password: libliblib
24
25 student credentials:
26 email: student@gmail.com
27 password: studentstudent
```

B Software Security Code of Practice Self-Assessment Template

Self-Assessment Details	
Organisation:	Streeling University Library
Assessed (Name): by	Student ID: u5602780
Role:	Author / Developer
Date Completed:	January 23, 2026

Table 4: Software Security Code of Practice Self-Assessment

Reference	Principles / Claims	Evidence	Arguments / Assumptions
Theme 1: Secure design and development			
Principle 1.1	Follow an established secure development framework		
Claim 1.1.1	The development framework used is documented.	Section 3 documents a "DevSecOps methodology" divided into 4 specific phases (Alpha, Robust Expansion, Cleanup, Cosmetic).	The report outlines a structured approach to development integrating security from the outset.
Claim 1.1.3	Tools are maintained and updated.	All development tools, libraries, and frameworks are pinned to their latest stable versions to mitigate known vulnerabilities.	It is assumed that up-to-date versions are maintained by the package managers.
Claim 1.1.4	Items requiring configuration control are identified and version control is used.	The GitLab environment provided by the University department is utilised.	
Continued on next page			

Table 4 – continued from previous page

Reference	Principles / Claims	Evidence	Arguments / Assumptions
Claim 1.1.5	Requirements are captured and recorded.	Section 1 explicitly lists Frontend requirements (MVP) and provides a table of Functional and Non-Functional requirements / User Stories.	Requirements were derived from the goal of separating concerns between students and staff.
Claim 1.1.6	Software is designed for user need.	Section 1 outlines specific needs for Students (Dashboard, Catalogue) and Librarians (Inventory Panel, Audit Log).	X
Principle 1.2	Understand the composition of the software and assess risks linked to the ingestion and maintenance of third-party components throughout the development lifecycle		
Claim 1.2.1	All third-party components are identified and documented.	X	X
Claim 1.2.2	Integrity of third-party components and updates is verified.	X	X
Claim 1.2.3	Each third-party component is tested before being first deployed.	X	X
Claim 1.2.4	Third-party component updates are tested.	Lates versions are installed react 19	X
Claim 1.2.5	Processes are in place to manage and deploy updates to third-party components.	X	X
Principle 1.3	Have a clear process for testing software and software updates before distribution.		
Continued on next page			

Table 4 – continued from previous page

Reference	Principles / Claims	Evidence	Arguments / Assumptions
Claim 1.3.1	A test plan exists that covers all requirements and third-party components.	Phase 3 (Section 3) mentions "Advanced tests" and "automated api/frontend testing".	X
Claim 1.3.2	Execution of the test plan is automated and repeatable wherever possible.	Section 3 Phase 3 explicitly lists "automated api/frontend testing".	X
Claim 1.3.3	Defects identified during testing are addressed.	Section 4 "Critical Review" details specific defects (e.g., Asynchronous State Rendering, Boolean Logic) and the code refactors used to fix them.	Evidence shows an iterative process where identified issues were resolved.
Principle 1.4	Follow secure by design and secure by default principles throughout the development lifecycle of the software.		
Claim 1.4.1	Techniques to understand how the software might be exploited (threat modelling) have been used in the design of the software.	Section 2 defines a "Zero Trust Client" approach, treating the frontend as a "glass box" vulnerable to inspection.	The design assumes the client is compromised by default.
Claim 1.4.2	Multi-factor authentication for privileged users of the software is enforced.	X	X
Claim 1.4.3	Default (and persistent) passwords are not used.	X	X
Claim 1.4.4	Data input to the software is validated.	Section 2 states "validation is performed twice: once on the frontend... and rigorously on the backend".	Adopts a "Defence in Depth" strategy for validation.
Continued on next page			

Table 4 – continued from previous page

Reference	Principles / Claims	Evidence	Arguments / Assumptions
Claim 1.4.5	Credentials and sensitive data are securely stored.	Section 3 details use of HttpOnly cookies with Same-Site='Lax' to protect tokens.	Encryption at rest is listed as a "Future Improvement," implying it is not currently implemented.
Theme 2: Build environment security			
Principle 2.1	Protect the build environment against unauthorised access.		
Claim 2.1.1	Roles are defined that specify the data and functionality that each role is allowed to access.	X	Report details RBAC for the application users, but not for the build environment/developers.
Claim 2.1.2	Users of the build environment are required to authenticate on a regular basis.	The lifetime of the login cookie is set to 1 hour, forcing periodic re-authentication.	X
Claim 2.1.3	Users of the build environment are issued with credentials bound to their role.	RBAC is implemented and cookies have role information	X
Claim 2.1.4	Credentials are securely managed and stored.	All passwords are hashed, and there is no direct external access to the database.	X
Claim 2.1.5	Credentials are multi factor.	X	X
Principle 2.2	Control and log changes to the build environment.		
Claim 2.2.1	Access and changes to the build environment are logged.	X	X
Claim 2.2.2	Only authorised personnel can make changes to the build environment.	X	X
Continued on next page			

Table 4 – continued from previous page

Reference	Principles / Claims	Evidence	Arguments / Assumptions
Claim 2.2.3	Logs are auditable and retained for an agreed period.	X	Audit logs exist for the Application (Section 3.1), but not specified for the Build Environment.
Claim 2.2.4	The confidentiality and integrity of logs is protected.	X	X
Theme 3: Secure deployment and maintenance			
Principle 3.1	Distribute software securely to customers		
Claim 3.1.1	The integrity of software (including updates) can be verified in the customer environment.	X	X
Claim 3.1.2	Software (including updates) is distributed over trusted channels.	Gitlab is used as main distribution	X
Principle 3.2	Implement and publish an effective vulnerability disclosure process.		
Claim 3.2.1	A vulnerability disclosure policy and process is published.	X	X
Claim 3.2.2	The vulnerability disclosure process describes how to confidentially report vulnerabilities.	X	X
Principle 3.3	Have processes and documentation in place for proactively detecting, prioritising and managing vulnerabilities in software components.		
Claim 3.3.1	Knowledge of public vulnerabilities is kept up to date.	X	X
Claim 3.3.2	A vulnerability management plan exists that assesses and prioritises responses to vulnerabilities.	X	X
Principle 3.4	Report vulnerabilities to relevant parties where appropriate.		
Claim 3.4.1	Internal security teams are informed	X	X
Continued on next page			

Table 4 – continued from previous page

Reference	Principles / Claims	Evidence	Arguments / Assumptions
Claim 3.4.2	Affected customers are informed	X	X
Principle 3.5	Provide timely security updates, patches and notifications to customers.		
Claim 3.5.1	Security updates are distributed as soon as is practicable.	X	X
claim 3.5.2	Security updates are tested and secure by default.	X	X
Theme 4: Communication with customers			
Principle 4.1	Provide information to the customer specifying the level of support and maintenance provided for the software being sold.		
Claim 4.1.1	‘End of support’ dates are published for all software components.	X	X
Claim 4.1.2	A policy on frequency of updates and the process for applying them is published.	X	X
Claim 4.1.3	User documentation describes how to correctly and securely apply updates and use software.	X	X
Principle 4.2	Provide at least 1 year’s notice to customers of when the software will no longer be supported or maintained by the vendor.		
Claim 4.2.1	Customers are given at least 1 year’s notice of when software will no longer be supported.	X	X
Principle 4.3	Make information available to customers about notable incidents that may cause significant impact to customer organisations.		
Claim 4.3.1	Customers are informed of relevant incidents in a timely manner	X	X

C Appendix: GAIT Usage

GAIT tools were used for:

1) testing against requirements: prompt: Analyse this work against attached requirements and marking criteria. Be strict and honest appendix is not marked think a lot. Gemini surprisingly good at it

2) learn react

more than half of the time spent on the project was to learn basics of react, javascript

prompt: I want you to be my react teacher. Without providing copy paste code that i can use. I need to learn everything myself

This was hit or miss. It was good for the basics, but in the end, it gave odd suggestions that broke the code and reduced its quality Thankfully, there is enough material online to learn those principles from blogs

3) All sorts of help with latex formatting, where ChatGPT was really helpful